

Embodied RL Roadmap: 从经典控制到 Diffusion/VLA 后训练

资料截止：2026-09-12

摘要

具身智能中的强化学习 (reinforcement learning, RL) 经历了从状态/动作空间中的经典控制, 到深度策略、仿真到现实 (sim-to-real)、真实机器人在线/离线 RL, 再到扩散策略、视觉语言动作模型 (VLA) 后训练的演化。主线并不是“PPO 换成了某个新算法”, 而是三件事一起变化: 反馈从手工奖励扩展为示范、偏好、任务成功与世界模型反馈; 策略从单峰高斯扩展到动作序列的多模态生成; 数据从少量人工轨迹扩展为仿真、离线数据和真实机器人数据飞轮。

本文用 Flow-GRPO 作为扩散模型在线 RL 的前置, 用 DiffusionNFT 作为 forward-process 后训练的代表, 并分析这些图像生成方法迁移到具身动作生成时哪些结构可以复用、哪些地方必须重新设计。图像生成的奖励提升不能直接推出机器人闭环控制提升, 这一限制贯穿全文。

0. 给初学者的最短预备课

可以先把 RL 想成“让一个会试错的控制器学会做事”。机器人看到相机画面和自身关节状态, 这些信息组成状态 s_t ; 控制器输出动作 a_t , 例如关节速度或夹爪开合; 环境执行动作后给出下一个状态 s_{t+1} 和奖励 r_t 。一段从开始到结束的交互记录叫作轨迹 (trajectory)。如果机器人把杯子放到目标位置, 最后可能得到较高奖励; 如果撞倒杯子, 则得到低奖励或惩罚。

这里最容易混淆的三个词是: ** 策略 (policy) ** 是“看到状态后如何选动作”的规则, ** 奖励 (reward) ** 是某一步得到的即时分数, ** 回报 (return) ** 是从当前时刻往后的折扣奖励总和。RL 的学习目标不是记住某一个动作, 而是让策略在未来许多步之后仍能取得较高回报。因此, 长任务中的“信用分配”很重要: 如果 20 步之后才知道任务失败, 算法要判断前面哪几步导致了失败。

具身智能又比游戏或图像生成多一层困难: 动作真的会改变世界。图像模型可以一次生成很多候选图片, 再由评分器打分; 机器人每次探索都要花时间, 还可能碰撞、磨损或需要人工重置。后文出现“在线”时, 指策略与环境持续交互; 出现“离线”时, 指只从已经收集好的轨迹数据学习。

1. 先建立统一坐标系

在普通 MDP 中, 策略 $\pi(a_t|s_t)$ 根据状态选择动作, 奖励 r_t 评价短期行为, 价值函数负责长时序信用分配。具身任务额外包含视觉观测、动力学、接触、延迟、安全约束和硬件磨损。因此同一个“在线 RL”在图像生成和机器人控制中并不等价: 图像可以在离线评分器上快速反复采样, 机器人每一次探索都可能消耗时间、撞击物体或损坏硬件。

本文用六个维度比较方法: 反馈来源、策略表示、训练环境、在线程度、信用分配粒度和部署约束。

2. 发展脉络

2.1 经典 RL 与机器人控制: 从值函数到策略优化

早期机器人 RL 主要处理低维状态、动作和可设计奖励。Q-learning 学习“在某个状态做某个动作值不值得”, actor-critic 同时学习策略和价值估计, policy gradient 直接调整策略参数, PPO 则用裁剪后的更新限制策略一次变化太大。它们解决的是不同的优化稳定性问题, 但都需要回答同一个问题: 怎样从奖励反推出哪些动作值得重复。

真正进入复杂机器人后, 难点从“有没有 RL 算法”变成样本效率、探索安全和奖励塑形。比如“把物体放进盒子”可以把成功作为稀疏奖励, 但机器人可能很久都碰不到盒子; 把距离、接触力和姿态加入奖励会更容易学, 却可能诱导策略钻奖励函数的空子。这就是为什么具身 RL 通常把示范学习、课程学习、奖励模型和安全约束放在同一个系统里。

2.2 深度 RL 与 Sim-to-Real: 把探索移入仿真

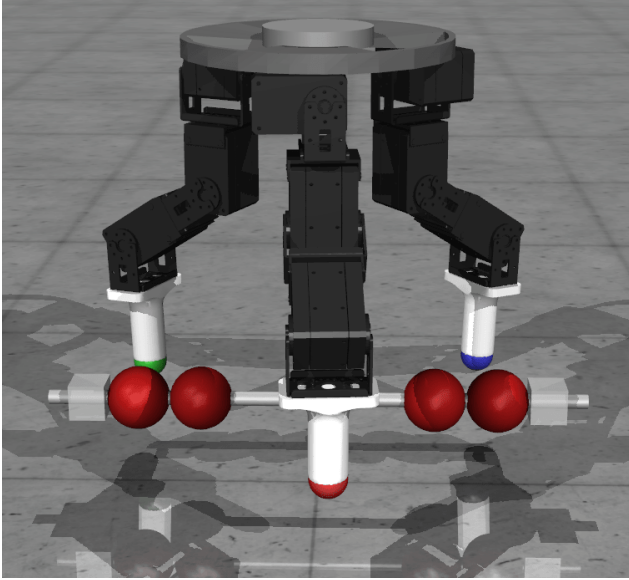
深度视觉策略、域随机化、系统辨识和残差控制把大部分探索移到仿真中。其典型流程是: 在仿真中用大量并行环境训练策略, 再通过视觉、动力学和执行器随机化迁移到真实机器人。域随机化不是把仿真做得“看起来漂亮”, 而是主动改变光照、相机噪声、摩擦、质量、延迟和控制频率, 让策略学到对这些变化不敏感的规律。

这条路线的关键假设是: 真实世界落在训练时覆盖的变化范围内。优点是样本量可扩展, 缺点是仿真偏差会集中出现在接触、摩擦、遮挡和长时序任务上。一个在仿真中成功率很高的策略, 仍可能因为真实夹爪略有偏差而抓不稳。因此 sim-to-real 往往还需要少量真实数据做校准或残差修正。

2.3 真实世界 RL：反馈闭环成为系统工程

The Ingredients of Real-World Robotic Reinforcement Learning 将问题拆成奖励、探索、数据采集、重置、硬件安全和持续训练等组件。这里的“ingredients”可以理解为一套运行条件：奖励要能自动计算，机器人要能在失败后回到可工作的初始状态，数据要记录传感器和动作时间戳，控制器要有速度/力矩边界，训练过程还要能筛掉异常轨迹。

真实世界 RL 由此从单一算法问题转为完整系统问题。自动重置、并行机器人、失败恢复和数据过滤与优化器同样重要。很多论文的性能提升来自整个数据闭环，而不是某一个损失函数单独带来的提升；阅读结果时要把“算法贡献”和“系统工程贡献”分开。



2.4 离线 RL 与示范学习：先从已有经验中学习

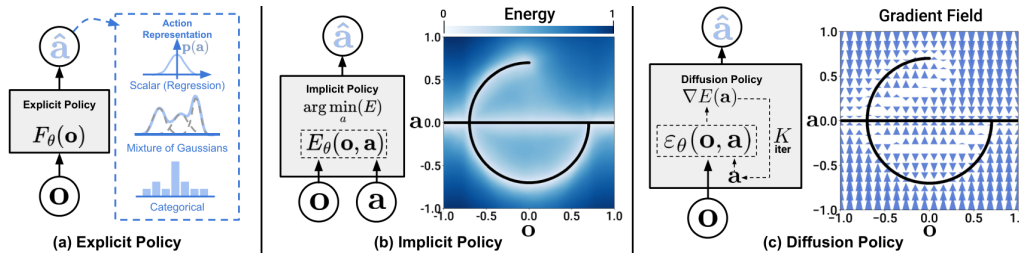
离线 RL 通过固定数据集学习策略，避免一开始就在真实世界探索。数据可以来自专家示范、旧策略、遥操作或过去的失败尝试。D4RL、IQL、CQL、AWAC 和 diffusion-based offline RL 代表了不同路径。离线数据让策略学习更安全，但也带来两个初学者需要特别注意的问题：第一，数据里没有出现过的动作很难可靠评估；第二，数据只记录了发生过的结果，不能自动告诉模型“如果当时换个动作会怎样”。

机器人实践通常采用“示范/离线预训练 + 少量在线改进”，而不是纯离线或从零在线。前者提供一个不会立刻摔倒的初始策略，后者利用真实奖励修正示范中的盲点。

2.5 Diffusion Policy：把动作看成多模态序列

Diffusion Policy 将动作序列作为条件生成对象，使用视觉观测和历史动作预测一段 action chunk。这里的“扩散”可以先用一个直观比喻理解：训练时逐渐给真实动作加噪声，模型学习如何一步步去噪；执行时从噪声开始，反过来逐步生成一段动作。因为生成的是分布而不是一个平均动作，它可以保留多个同样合理的行为模式。

例如，同一个物体可以从左边抓，也可以从右边抓。普通回归模型容易把两条轨迹平均成一条不可执行的中间轨迹；扩散策略可以把它们作为两个模式保留下来。代价是执行时要做多步采样，而且每次生成的动作仍需放回真实闭环中检查。



Diffusion Policy 首先是策略表示和监督/模仿学习方法，不自动等于 RL。要把它用于 RL，需要定义任务奖励、探索机制、动作序列信用分配和安全约束。Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning 进一步说明扩散分布可以作为离线 RL 的策略类，但仍受数据覆盖和价值外推影响。

3. Flow-GRPO: DiffusionNFT 的必要前置

3.1 GRPO 的直觉

对同一个条件 c 采样 N 个结果 $x^{(i)}$ ，计算每个结果的奖励 $R^{(i)}$ ，再用组内均值和标准差得到相对优势：

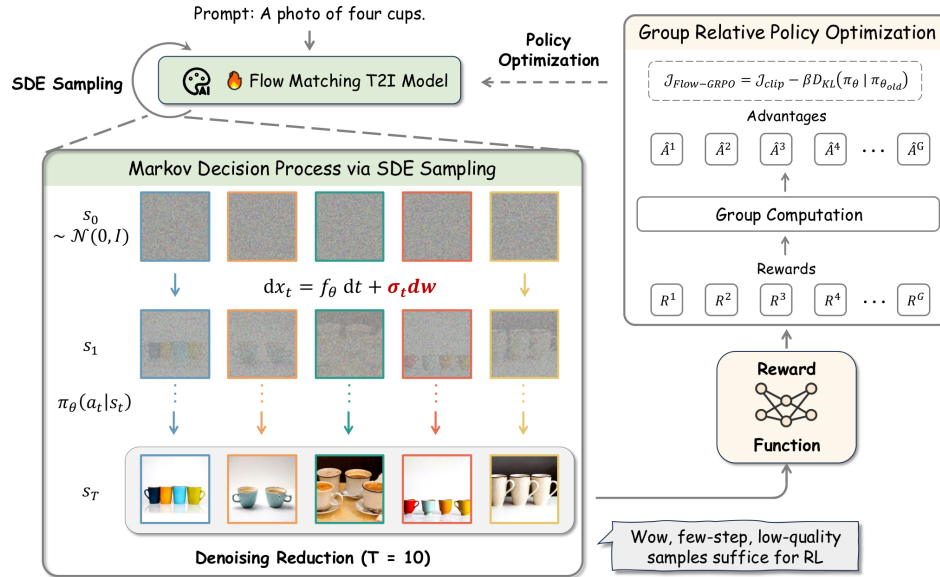
$$A^{(i)} = \frac{R^{(i)} - \text{mean}(R)}{\text{std}(R) + \epsilon}.$$

高于组平均的结果被加强，低于组平均的结果被削弱。它不需要单独训练一个 value model，因此适合把奖励直接用于生成模型后训练。

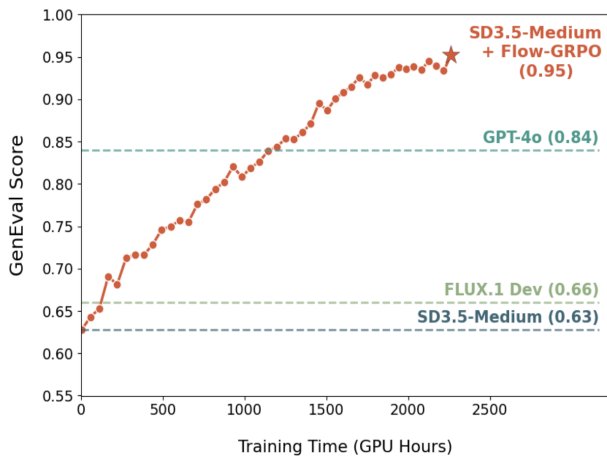
举一个很小的例子：同一个指令生成 4 张图，评分为 0.2、0.4、0.7、0.8。GRPO 不需要知道“绝对的 0.8 是否足够好”，只需要知道后两张比组内平均更好，于是提高产生它们的概率。对机器人而言，同一观测也可以采样多段候选动作，用任务成功、距离或碰撞惩罚进行组内比较。难点在于机器人奖励通常延迟到整段动作结束，信用分配会比单张图像更难。

3.2 为什么 flow matching 需要额外处理

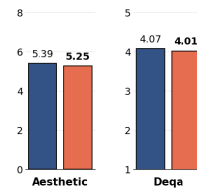
许多 flow-matching 采样器由确定性 ODE 描述。确定性路径没有传统 policy gradient 所需的随机探索，也不直接给出每一步的动作概率。Flow-GRPO 的关键步骤是把 ODE 转换为保持边际分布的 SDE，使每一步拥有可计算的随机转移；随后对同一 prompt 的多条生成轨迹计算组相对优势，并把最终图像奖励回传到去噪过程。



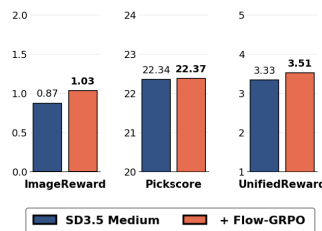
(a) GenEval Performance



(b) Image Quality



(c) Preference Score



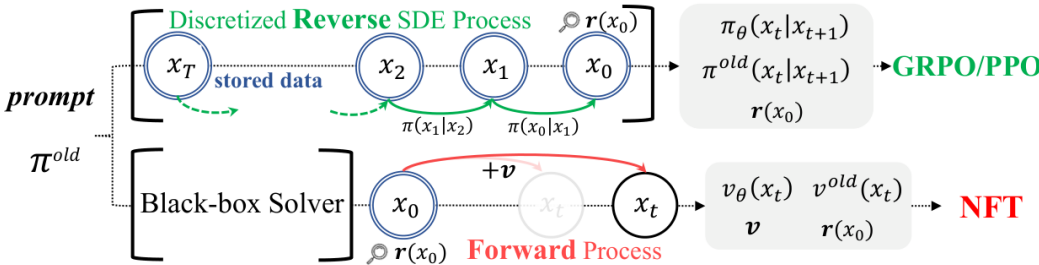
Flow-GRPO 的优点是首次把在线策略梯度式 RL 连接到 flow-matching 生成模型；代价是仍要处理反向采样轨迹、概率比、solver 约束和逐步信用分配。论文报告其在文本到图像任务上改善组合关系、文字渲染和偏好对齐，但这些指标属于图像生成证据，不是具身控制证据。

可以把它理解成两层循环：外层对同一个条件采样多条完整生成路径并比较奖励，内层沿每条路径的多个去噪步骤更新模型。外层回答“哪张结果更好”，内层回答“这条结果中的哪些随机转移应该更常发生”。Flow-GRPO 的工程困难正来自这条内层路径：需要保留或重建每一步的随机转移，并让训练时的概率计算和实际采样 solver 对得上。

4. DiffusionNFT: 从 reverse trajectory 到 forward process

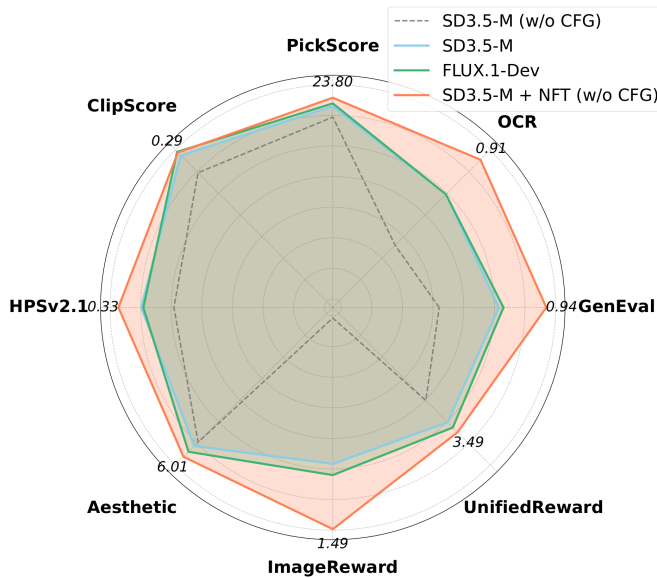
4.1 方法定位

DiffusionNFT (Diffusion Negative-aware Fine-Tuning) 提出在 forward diffusion / flow-matching 过程中直接做在线 RL。它把生成结果按奖励划分为正样本和负样本，用二者的预测误差构造隐式的策略改进方向，并把 RL 信号整合进类似监督学习的 velocity objective。



与 Flow-GRPO 的对照可以概括为：

维度	Flow-GRPO	DiffusionNFT
优化位置	reverse sampling trajectory	forward/noising 与 flow-matching objective
探索形式	ODE-to-SDE 后的随机轨迹	生成样本的正负对比
轨迹存储	需要逐步采样信息	只需 clean samples 参与训练
solver	对采样器有更强约束	支持黑盒 solver
CFG	需要处理 CFG/参考策略	论文强调 CFG-free 设定
直观理解	对轨迹做 policy gradient	用奖励排序改变 velocity field



论文报告在 SD3.5-Medium 上相对 Flow-GRPO 的效率优势，例如 GenEval 从 0.24 提升到 0.98 的对比；这些数字应理解为图像生成实验中的 paper fact，不能直接外推到机器人成功率。

对初学者来说，DiffusionNFT 的核心变化可以浓缩成一句话：Flow-GRPO 把“整条反向生成轨迹”当作 RL 的对象，DiffusionNFT 则尽量把奖励信息转换成 forward-process 上的训练信号。模型仍然需要生成样本并获得奖励，但优化阶段主要比较好样本和坏样本在 velocity prediction 上的差异。这样做减少了对完整采样轨迹和显式似然估计的依赖。

这并不意味着它不需要探索或奖励。它仍然需要候选样本、奖励模型和正负划分；改变的是“如何把奖励变成参数更新”。因此论文中“只需 clean samples”应理解为训练更新不必保存整条反向轨迹，而不是完全不需要生成过程。

4.2 迁移到具身动作生成：哪些东西可以复用

图像生成	具身智能中的对应物
prompt	语言指令、视觉状态和任务上下文
clean image	action chunk 或动作轨迹
image reward	任务成功率、距离、碰撞、能耗、平滑度
正/负图像	成功/失败 rollout 或候选动作
forward noising	动作扰动、latent/action flow
单次生成质量	长时序闭环控制表现

可迁移的结构包括正负样本比较、forward-process 优化和 solver 解耦。**必须重做的部分**包括时间信用分配、动作连续性、动力学可行性、在线数据飞轮、碰撞和安全过滤、sim-to-real 校准。机器人动作是带动力学的时间序列，单个最终动作 chunk 的奖励无法自动告诉模型哪一个去噪时刻或哪一步接触导致了失败。

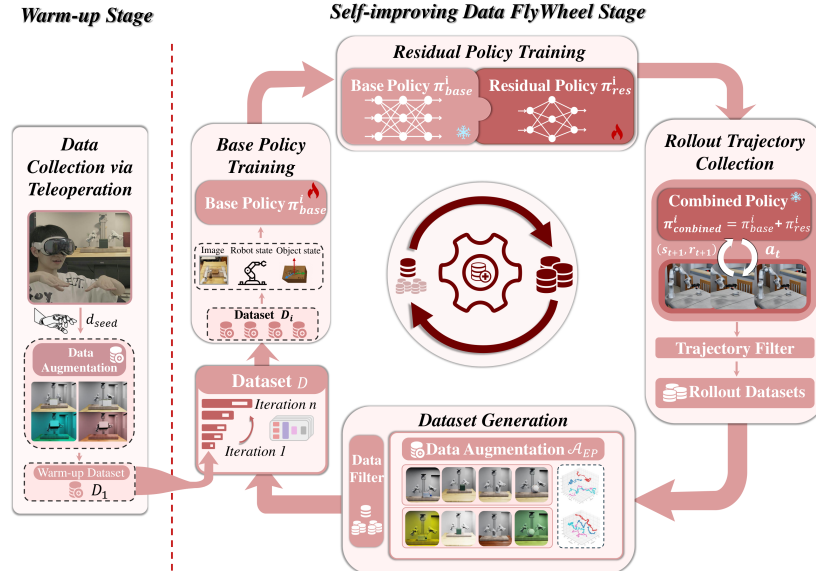
因此，DiffusionNFT 更适合作为“扩散动作策略后训练的候选机制”，而不是已经完成验证的机器人算法。具身实验需要至少增加：仿真 benchmark、真实机器人闭环成功率、不同 embodiment 的迁移、失败率/碰撞率、动作平滑度和数据效率。

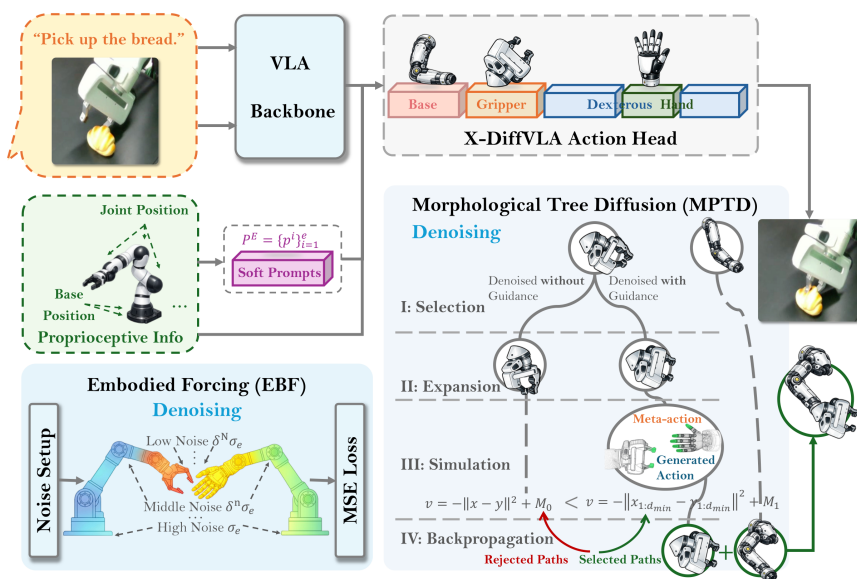
一个可执行的迁移实验是：先固定一个已经用示范训练好的 diffusion action policy，在仿真中为同一观测生成一组 action chunks；用任务成功、末端距离和碰撞惩罚排序；再比较普通行为克隆、轨迹级 Flow-GRPO 风格更新和 forward-process/NFT 风格更新。只有当三种方法使用相同的采样预算、奖励函数和安全过滤，并在未见过的初始位姿上测试，才能判断 NFT 风格更新是否真的带来数据效率提升。

5. 从动作扩散到 VLA 与数据飞轮

OpenVLA 等开放 VLA 模型把视觉、语言和动作统一到一个策略中。扩散/flow action head 负责连续动作生成，语言视觉编码器负责任务条件。此时 RL 的角色从“从零学策略”逐渐转为：在示范预训练策略上做任务后训练、在线纠错和奖励对齐。

DexFlyWheel 将灵巧操作的数据采集、失败分析和策略改进组织成可循环的数据框架；RL-100 则将真实机器人数据飞轮、离线 RL 和在线提升组合起来，代表“先利用人类先验，再用真实反馈超越示范”的路线。





近期工作开始把 flow/diffusion action head 与跨机器人 embodiment 迁移结合起来，但仍要区分“动作头的表达能力”“VLA 的语义泛化”和“真实机器人 RL 的闭环适应”。

6. 按阶段比较

阶段	主要数据	策略表示	RL 的主要作用	主要瓶颈
经典控制	状态、少量轨迹	值函数/高斯策略	学控制规律	状态维度和奖励设计
深度 RL + 仿真	大规模仿真 rollout	深度策略	探索与控制	sim-to-real
真实世界 RL	机器人在线数据	深度策略/残差策略	适应真实反馈	样本效率、安全
离线 RL	示范与历史数据	Q/actor/diffusion	从固定数据改进	OOD 动作、覆盖不足
Diffusion Policy	示范动作序列	扩散动作分布	表达多模态动作	推理成本、闭环校正
Flow-GRPO/DiffusionNFT	生成结果与奖励	flow/diffusion 生成器	奖励对齐与后训练	奖励黑客、信用分配
VLA + RL	视觉语言动作数据飞轮	foundation policy + action head	任务后训练、迁移	长时序、硬件和泛化

7. 结论与研究问题

1. **反馈闭环比算法名称更关键**：没有可靠的奖励、重置、过滤和失败记录，换成更复杂的 RL 目标也无法稳定提升机器人能力。
2. **扩散策略解决的是分布表达问题**：它可以表达多模态动作，但在线 RL 还需要解决动作序列信用分配和安全探索。
3. **Flow-GRPO 与 DiffusionNFT 是两种互补路线**：前者把轨迹级策略梯度接到 flow sampler，后者把奖励排序转成 forward-process 的监督式改进。
4. **图像生成结果不能直接证明具身迁移**：迁移假设必须由仿真、真实机器人和跨 embodiment 实验验证。
5. **RL-100 类系统提示了落地路线**：人类示范/预训练策略提供先验，真实机器人数据飞轮提供持续反馈，离线 RL 提高安全性，在线 RL 完成最后的任务适应。

8. 阅读与复现顺序

1. Diffusion Policy: 理解动作扩散策略表示;
2. Flow-GRPO: 理解 ODE-to-SDE、组相对优势和轨迹级 RL;
3. DiffusionNFT: 理解 forward-process、正负样本和 solver 解耦;
4. Ingredients / DexFlyWheel / RL-100: 理解真实机器人系统和数据闭环;
5. OpenVLA / X-DiffVLA: 理解 VLA、扩散动作头和跨 embodiment;
6. 在仿真环境中比较 BC、Diffusion Policy、Flow-GRPO 风格更新和 NFT 风格更新，再考虑真实机器人。

参考链接

Flow-GRPO 论文·代码; DiffusionNFT 论文·代码; Diffusion Policy; RL-100; DexFlyWheel; OpenVLA; X-DiffVLA; Diffusion Models for RL Survey.